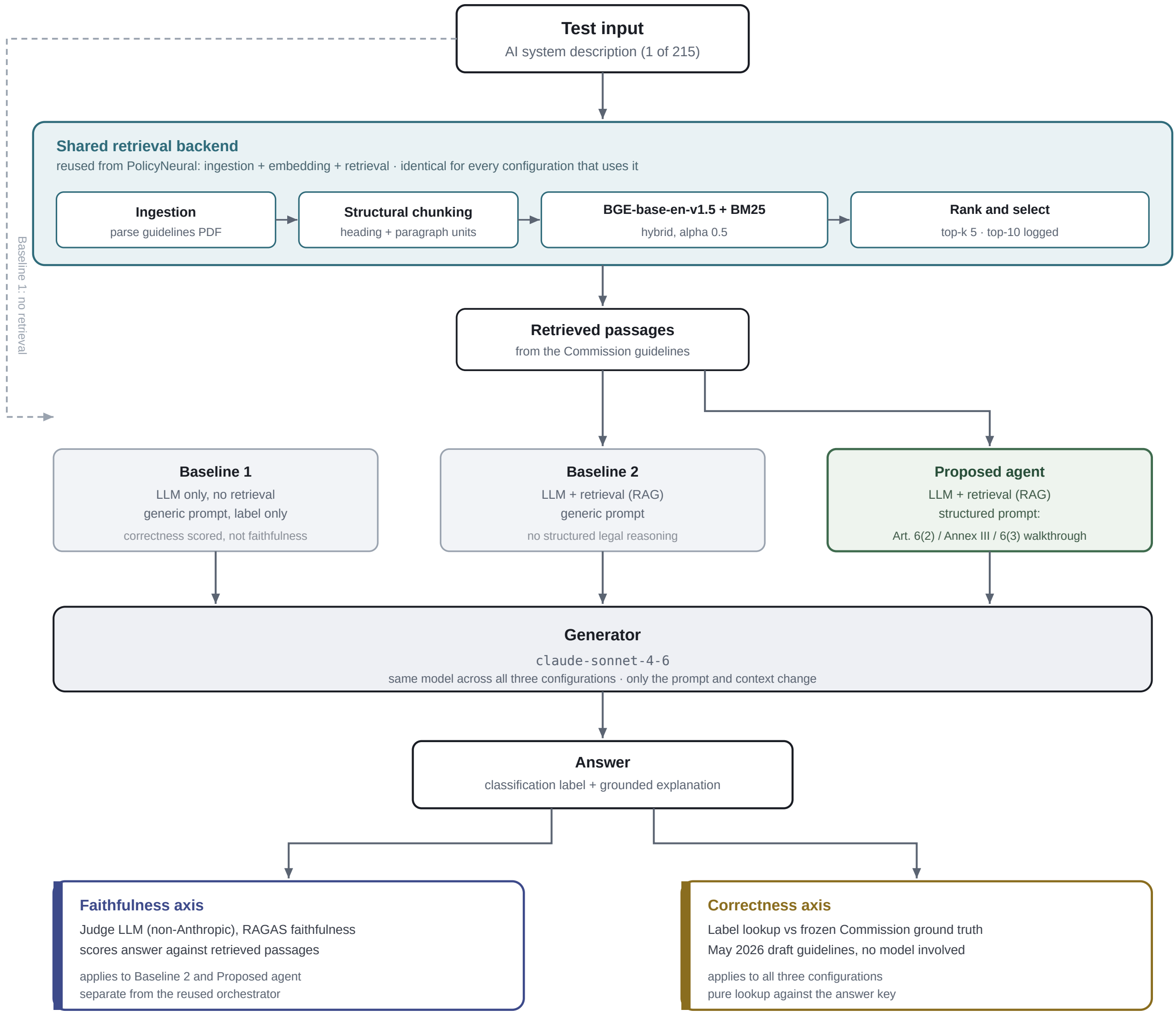
Agent architecture and component selection

Case Study 2 · EU AI Act high-risk classification, evaluation of RAG faithfulness against legal correctness

Rahul Rawat · 100004438 · MSc Data Science, SRH Heidelberg · Supervisor: Prof. Swati Chandna · 25 August 2026

This note documents the system architecture for Phase 2 and records which parts of the reused PolicyNeural project are kept and which are deliberately left out. The compliance agent is the system under test, not the deliverable, so the design is driven by one requirement: the pipeline must fetch text and classify, and it must do nothing that generates, corrects, or grades that classification inside itself. Anything that violates that would contaminate the two things the study measures separately, whether the answer is grounded in the retrieved passages (faithfulness) and whether the legal label is right (correctness).

FIGURE 1 · EVALUATION ARCHITECTURE



Reading it: one test input flows through the shared retrieval backend, then the same generator answers it under three prompt configurations. The answer is then measured on two independent axes. Faithfulness and correctness are scored by different mechanisms and neither is allowed to influence the other, which is what makes the divergence between them measurable. Baseline 1 skips retrieval, so only correctness applies to it.

FIGURE 2 · WHAT IS KEPT FROM POLICYNEURAL AND WHAT IS NOT

USED · reused as infrastructure (3)

- **Ingestion**
  loads and parses the Commission guidelines
- **Embedding**
  BGE-base-en-v1.5, open source, CPU friendly
- **Retrieval**
  hybrid BM25 + dense, alpha 0.5, top-k 5

EXCLUDED · deliberately not used (9)

- DebateAgent A
- DebateAgent B
- VerifierAgent
- EvaluatorAgent
- SummarizerAgent
- MemoryAgent
- TopicModelAgent
- PlannerAgent
- GuardrailsAgent

Reason category

- confounds the faithfulness measurement
- breaks reproducibility
- not relevant to this task

Nine of PolicyNeural's twelve components are dropped. The three kept components all fetch or index text. Every dropped one either produces, corrects, grades, or gates an answer, or has nothing to do with the task.

WHY EACH EXCLUDED COMPONENT IS EXCLUDED

COMPONENT	REASON FOR EXCLUSION
WOULD CONFOUND THE FAITHFULNESS MEASUREMENT	
`DebateAgent A` `DebateAgent B`	They run competing generation passes and reconcile them, so there is no single clean answer to score. The study needs exactly one answer per input from one generator, otherwise it is unclear which text RAGAS is even judging.
`VerifierAgent`	Adds a correction pass over the generator output. That would quietly repair wrong labels and erase the faithful but wrong gap this study exists to find. If a verifier fixes the errors before they are scored, the headline result cannot occur.
`EvaluatorAgent`	Scores the answer from inside the pipeline. Faithfulness is already scored by a separate RAGAS judge in a different model family, so an internal evaluator would be the generator marking its own work, which the design specifically avoids.
`SummarizerAgent`	Compresses the retrieved passages before generation. The generator would then read a summary while RAGAS scores the answer against the full passages. The text the generator reads and the text the judge scores against have to be identical, or the faithfulness number is meaningless.
WOULD BREAK REPRODUCIBILITY	
`MemoryAgent`	Adapts retrieval for each row based on earlier rows, so results depend on the order the rows are processed and cannot be reproduced. It also breaks the rule that all three configurations see identical retrieval, which is the only reason the three are comparable at all.
NOT RELEVANT TO THIS TASK	
`TopicModelAgent`	Finds latent topics across a corpus. The task is a fixed classification against Annex III, not topic discovery, so it has nothing to contribute.
`PlannerAgent`	Decomposes a task into steps at runtime. The legal test here is fixed and known in advance, and the structured reasoning already lives in the proposed agent prompt as a set Article 6(2), Annex III, 6(3) walkthrough. A dynamic planner would only add variation we cannot control.
`GuardrailsAgent`	Filters or rewrites inputs and outputs for safety. The inputs are benign system descriptions and the study needs the raw classification, including the wrong ones. A guardrail that suppresses or rewrites outputs would hide the very errors being measured.

> **Net effect.** The pipeline keeps only the parts that fetch and index text and nothing that generates, corrects, grades, or gates the classification. That is what lets the faithfulness score and the correctness check stay independent. Their independence is the object of study, so anything that couples them is removed by design, not by omission.